

Maratona de Programação Paralela

ERAD/RS – 2026

cradrs.github.io/eradrs2026

Calebe P. Bianchini¹ e Matheus S. Serpa²

¹Universidade Presbiteriana Mackenzie
CESAR - Centro de Estudos e Sistemas Avançados do Recife
linkedin.com/in/calebebianchini

²Universidade Federal do Rio Grande do Sul
linkedin.com/in/matheusserpa

Regras Gerais

Leia com atenção as seções de entrada e saída de cada problema. Para cada problema, será fornecida uma implementação sequencial de referência. A saída do seu programa será comparada com a saída dessa implementação para decidir se a sua solução está correta. Você pode modificar o programa da maneira que achar adequada, exceto quando a descrição do problema indicar o contrário.

Você deve enviar um arquivo compactado (*zip*) com seu código-fonte, o *Makefile* e um script de execução. O script deve ter o nome do problema. Você pode enviar quantas soluções para um problema desejar; somente a última submissão será considerada. O *Makefile* deve possuir a regra `all`, que será utilizada para compilar seu código-fonte. Esse script executa sua solução da maneira que você a projetou e será inspecionado para verificar se não prejudica o ambiente de julgamento.

A maratona avalia programação paralela. Para obter pontuação, a solução submetida deve explorar paralelismo de forma efetiva, por exemplo com múltiplas threads, processos, nós ou GPU. Otimizações sequenciais são permitidas como complemento, mas não substituem a exigência de paralelismo. Soluções que contenham apenas otimizações sequenciais, mesmo que sejam mais rápidas que a implementação de referência, serão desclassificadas. A organização poderá inspecionar o código-fonte e o script de execução para verificar o uso efetivo de paralelismo.

O tempo de execução do seu programa será medido com o programa *time*, considerando o tempo real decorrido. Cada programa será executado pelo menos três vezes com a mesma entrada, e o tempo médio será levado em consideração. O programa sequencial fornecido será medido da mesma maneira. Em cada problema, você ganhará pontos correspondentes à divisão do tempo sequencial pelo tempo do seu programa (*speedup*). A equipe que somar mais pontos ao final da maratona será declarada vencedora.

Este caderno contém páginas introdutórias e 3 problemas. A seção de problemas começa no Problema A, com numeração própria.

Informações Gerais

Compilação

Você deve usar **CC** ou **CXX** dentro do seu *Makefile*. Tenha cuidado ao redefinir essas variáveis! Há um *Makefile* simples dentro do pacote do seu problema que você pode modificar. Exemplo:

```
FLAGS=-O3
EXEC=sum
CXX=g++

all: $(EXEC)

$(EXEC):
    $(CXX) $(FLAGS) $(EXEC).cpp -c -o $(EXEC).o
    $(CXX) $(FLAGS) $(EXEC).o -o $(EXEC)
```

Cada máquina de julgamento possui seu grupo de compiladores. Consulte-os abaixo e escolha bem ao escrever seu *Makefile*.

Máquina	Compilador	Comando
HOST	GCC	C = gcc C++ = g++
MPI	Open MPI	C = mpicc C++ = mpic++
GPU	NVIDIA CUDA	C = nvcc C++ = nvcc

Submissão

Informações gerais

Cada problema deve ter um script de execução com o mesmo nome do problema: A, B ou C. Há um script simples no pacote do problema que pode ser modificado. O sistema de julgamento compilará seu pacote pela regra **all** do *Makefile* e, em seguida, executará somente esse script, fornecendo a entrada pela entrada padrão e comparando a saída padrão com a saída da implementação sequencial.

O script deve preparar apenas o ambiente necessário para a sua solução, como número de threads, variáveis de ambiente, argumentos do programa ou chamada ao executável gerado. Ele deve assumir que será executado no diretório raiz da submissão. Não use caminhos absolutos da sua máquina, não leia entradas fixas de arquivos locais e não escreva a resposta em um arquivo separado. A resposta do problema deve ser emitida na saída padrão.

Antes de compactar a submissão, teste o pacote como o juiz testará: compile pela regra **all**, execute o script do problema com uma entrada conhecida e compare a saída obtida. Verifique também se o script tem permissão de execução, se necessário.

Se a sua solução usa OpenMP ou outra biblioteca baseada em threads, deixe claro no script quantas threads serão usadas. Escolha esse número de forma compatível com a estratégia paralela implementada e com os recursos disponíveis no ambiente de julgamento. Exemplo:

```
#!/bin/bash
# Exemplo: executa uma solução OpenMP com 32 threads
export OMP_NUM_THREADS=32
./sum
```

Submissão com MPI

Para submeter uma solução usando MPI, compile com `mpicc` ou `mpic++`. O script deve chamar `mpirun` ou `mpiexec` com o número escolhido de processos (máximo: 160). O sistema de julgamento prepara o ambiente de execução; portanto, não crie arquivos de máquinas, não dependa de nomes específicos de nós e não use parâmetros que só funcionem na sua máquina pessoal.

Escolha a quantidade de processos de acordo com a sua implementação paralela. Usar mais processos nem sempre melhora o desempenho; comunicação excessiva pode reduzir o *speedup*. Exemplo:

```
#!/bin/bash
# Exemplo: executa uma solução MPI com 4 processos
mpirun -np 4 ./sum
```

Comparando tempos e resultados

Na sua máquina pessoal, meça o tempo de execução da sua solução utilizando o programa *time*. Adicione redirecionamento de entrada/saída apenas ao testar localmente. Use o programa *diff* para comparar saídas textuais e *cmp* para comparar saídas binárias. O Problema A, por exemplo, usa entrada e saída binárias; nesse caso, uma comparação byte a byte é mais adequada.

Para saídas textuais, espaços, quebras de linha e mensagens extras podem fazer a comparação falhar. Seu programa deve imprimir somente o que o enunciado pede. Para saídas binárias, use uma comparação byte a byte. Exemplos de teste local:

```
$ time -p ./A < original_input.txt > my_output.txt
real 4.94
user 0.08
sys 1.56

$ diff my_output.txt original_output.txt
$ cmp my_output.bin original_output.bin
```

Não meça o tempo e **não** adicione redirecionamento de entrada/saída ao submeter sua solução – o *sistema de julgamento automático* está preparado para coletar seu tempo e comparar os resultados.

Problema A

N-corpos

Uma simulação de n corpos modela um sistema dinâmico de partículas, normalmente sob a influência de forças físicas, prevendo seus movimentos individuais. Uma aplicação conhecida de simulações de n corpos está na astrofísica, em que as partículas representam objetos celestes interagindo gravitacionalmente.

O código sequencial fornecido contém um simulador simples, mas funcional, de n corpos movendo-se em um espaço tridimensional. Sua tarefa é criar uma versão paralela deste simulador de n corpos, mantendo a correção da solução.

Entrada

A entrada consiste em um arquivo binário contendo, para cada partícula do sistema, uma estrutura com seis valores do tipo `float`. Esses valores representam a posição da partícula nos eixos x , y e z , e sua velocidade nos eixos x , y e z .

A entrada deve ser lida da entrada padrão.

Saída

A saída também é binária, no mesmo formato da entrada, contendo as posições e velocidades de cada partícula nos eixos x , y e z .

A saída deve ser escrita na saída padrão.

Exemplo – em forma textual, apenas para fins de leitura

Entrada exemplo	Saída exemplo
0.680375 -0.211234 0.566198 0.596880 0.823295 -0.604897 -0.329554 0.536459 -0.444451 0.107940 -0.045206 0.257742 -0.270431 0.026802 0.904459 0.832390 0.271423 0.434594 ...	-5.184253 11.632033 -11.737427 -63.642937 146.243469 -147.690735 10.654608 -15.666613 15.572982 133.827255 -195.720352 194.728912 22.627201 -1.267884 -5.422720 283.039612 -15.879736 -67.786301

Problema B

Centro da Árvore

Tiago A. O. Alves

A excentricidade de um nó u em um grafo $G(V, E)$ é a maior distância entre u e outro nó v , com $v \in V$. Em uma árvore, um centro é um nó com a menor excentricidade.

Veja um exemplo na Figura 1.

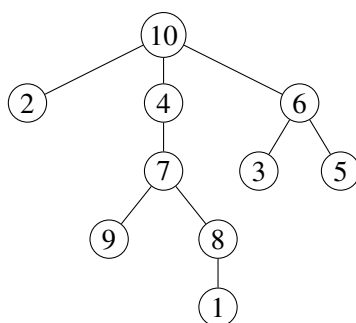


Figura 1. Uma árvore em que o centro é o nó 4, com distância máxima 3.

A entrada deste problema é uma árvore, e a saída deve ser o centro dela. Sua tarefa é desenvolver uma solução paralela para resolver este problema.

Entrada

A primeira linha da entrada contém um número inteiro N , que representa o número total de nós da árvore. As $N - 1$ linhas seguintes contêm as arestas não direcionadas entre os nós.

A entrada deve ser lida da entrada padrão.

Saída

A saída contém o rótulo do centro da árvore. Os casos oficiais terão centro único.

A saída deve ser escrita na saída padrão.

Exemplo

Entrada exemplo 1	Saída exemplo 1
10	4
10 4	
10 2	
10 6	
4 7	
6 3	
6 5	
7 9	
7 8	
8 1	

Problema C

Número de Submatrizes que Somam k

Leonardo M. Takuno

Dada uma matriz $A = (a_{ij})$ de inteiros, com dimensões $M \times N$, e um valor $k \in \mathbb{Z}$, este problema consiste em contar o número de submatrizes não vazias cuja soma é igual a k . A matriz pode conter números negativos. Uma submatriz é uma matriz obtida pela remoção de linhas e/ou colunas do início e/ou do fim da matriz original. Este problema é baseado no problema 1074 do LeetCode.

Por exemplo, dada a matriz com linhas $(1, 0)$ e $(0, 1)$, e $k = 1$, o resultado é 6: as duas submatrizes de um elemento com valor 1, as duas submatrizes de dimensão 1×2 , e as duas submatrizes de dimensão 2×1 .

Como segundo exemplo, dada a matriz com linhas $(1, -1)$ e $(-1, 1)$, e $k = 0$, o resultado é 5: as duas submatrizes de dimensão 1×2 , as duas submatrizes de dimensão 2×1 , e a submatriz de dimensão 2×2 .

Sua tarefa é desenvolver um programa paralelo para resolver este problema.

Entrada

A entrada contém apenas um caso de teste. A primeira linha contém três inteiros separados por espaço, nesta ordem: o valor k ($-10^8 \leq k \leq 10^8$), o número de linhas M e o número de colunas N da matriz A ($1 \leq M, N < 1000$). As M linhas seguintes contêm N inteiros cada, separados por espaço, representando os elementos a_{ij} de A ($0 \leq i < M$, $0 \leq j < N$, $-1000 \leq a_{ij} \leq 1000$).

A entrada deve ser lida da entrada padrão.

Saída

Imprima uma linha contendo um inteiro: o número de submatrizes cuja soma é k .

A saída deve ser escrita na saída padrão.

Exemplo

Entrada exemplo 1	Saída exemplo 1
1 2 2 1 -1 -1 1	2